Agentic Graph-RAG Research Analyst

Complete Architecture & Working — Including All Enhancements

Written in plain English. No jargon. Everything explained from scratch.

1. What Is This System?

Imagine you need to research a complicated topic — like how three pharmaceutical companies collaborated to make a vaccine, and how their factories around the world affected distribution. You could spend hours reading articles, taking notes, and trying to connect the dots.

This system does all of that for you automatically. You type a question. The system searches the web, reads the articles, figures out who is connected to who, builds a map of all those connections, and writes you a detailed answer with sources — all in under a minute.

But it goes further than just searching Google and summarising. It builds an actual relationship map (called a knowledge graph), understands the difference between a simple question and a complex one, and uses multiple strategies to make sure the answer is as good as possible.

> **In one sentence:** It is a smart research assistant that reads the internet, maps relationships between entities, and writes evidence-backed answers — and then checks its own work before giving you the result.

2. The Big Picture — How It Works End to End

Before diving into each part, here is the complete journey of a question through the system. Think of it like a factory assembly line where each station does one job and passes the result to the next.

1	**You type a question** The question enters the system. Before anything else happens, the system checks if the question is safe, reasonable, and within scope.
2	**The question is understood** The system figures out what kind of question it is — a simple definition, a question about a specific person or company, a question about relationships, or a time-based question. This decides how much work needs to happen.
3	**Routing — Fast or Deep?** Simple questions get a quick answer using just document search. Complex questions go through the full pipeline including knowledge graph construction.
4	**Web Search** The system searches the internet using multiple phrasings of your question to get a diverse set of sources.

5	**Documents are processed** Each article is read, split into chunks, and converted into numerical vectors so the system can find the most relevant pieces.
6	**Knowledge Graph is built** People, companies, events, and places are extracted from the documents. Relationships between them are mapped into a graph — like a mind map connecting everything.
7	**Graph is cleaned and scored** Old relationships are downweighted. Important entities are ranked using graph algorithms. The freshest, most relevant connections rise to the top.
8	**Memory is checked** The system checks if it has researched anything related before. If yes, that past knowledge is added to the current evidence.
9	**Three answers are generated in parallel** Using three different strategies, three independent answers are written simultaneously. A judge picks the best one.
10	**Quality is checked** The winning answer is scored on grounding, factuality, relevance, and completeness. If it fails, the system tries again automatically.
11	**Uncertainty is flagged** Any claim that is poorly supported by sources gets a warning marker added directly in the answer text.
12	**Answer is returned to you** The final answer, with inline citations and confidence flags, is returned. The session is saved to memory for future queries.

3. Input Guardrails — The Bouncer at the Door

Before the system does any work at all, it checks your question. Think of this as a security guard who checks everyone at the entrance. If your question is dangerous, manipulative, or completely out of scope, it gets rejected before any LLM call is made — saving time and cost.

What gets checked:

- **Length:** The question must be between 3 and 2000 characters. Too short means it is probably not a real question. Too long means it could be an attempt to overload the system.

- **Prompt Injection:** Phrases like 'ignore previous instructions' or 'you are now a different AI' are blocked. These are attempts to trick the system into doing something it should not.
- **Unsafe Content:** Questions asking how to make weapons or harm people are blocked immediately.
- **Malicious Intent:** Questions about spreading misinformation at scale or manipulating search algorithms are blocked.
- **Out of Scope:** The system is a research tool, not a personal assistant. Requests like 'book me a flight' or 'send an email' are rejected with a helpful message.
- **Rate Limiting:** Each user can send a maximum of 60 questions per minute. This prevents abuse.

> **Why this matters:** Without guardrails, a system like this could be exploited to generate propaganda, extract sensitive information, or crash under malicious load. The guardrails cost almost nothing (no LLM calls) and prevent all of this.

---

4. Understanding Your Question — Query Processing

Once the question passes the safety checks, the system needs to understand what you are really asking. This happens in two steps.

Step 1: Normalisation

The raw question is cleaned up. Extra punctuation is removed, whitespace is fixed, and the question is standardised. The system also does three things automatically:

- **Entity Detection:** Capitalised words are extracted as potential named entities — companies, people, places, products. So in 'How did Pfizer and BioNTech collaborate?', both Pfizer and BioNTech are extracted.
- **Time Detection:** Words like 'last year', 'recently', or '2023' are detected. This affects how the answer is cached and how fresh the sources need to be.
- **Domain Detection:** The question is categorised into a domain like technology, business, health, science, or politics. This helps route the question correctly.

Step 2: Intent Classification

The system makes one LLM call to classify what type of question you are asking. There are five types:

Intent Type	Example Question
SEMANTIC — asking what something is	'What is mRNA technology?'
ENTITY — asking about a specific thing	'Tell me about Moderna'
RELATIONAL — asking how things connect	'What is the relationship between Pfizer and BioNTech?'
TEMPORAL — asking about change over time	'How has vaccine distribution changed since 2020?'
HYBRID — a mix of the above	'Compare how OpenAI and Google approach AI safety'

This classification is critical because it determines everything downstream — which execution path to take, how long to cache the answer, and which retrieval strategies to prioritise.

5. The Router — Deciding How Much Work to Do

Not every question needs the full treatment. A simple question like 'What is quantum computing?' does not need a knowledge graph. A complex question like 'How do Pfizer's manufacturing partnerships affect global vaccine supply chains?' absolutely does.

The router makes this decision automatically based on the intent classification and a complexity score it calculates.

The Two Paths

FAST PATH	RESEARCH PATH
Used for simple, semantic queries	Used for relational, temporal, or complex queries
Vector search only — finds relevant documents by meaning	Full pipeline — vector search + knowledge graph construction
Takes 2-5 seconds	Takes 15-60 seconds
Lower cost — fewer LLM calls	Higher quality — deeper reasoning
Good for: definitions, simple facts	Good for: relationships, multi-hop reasoning, analysis

How Complexity is Scored

The router gives each question a complexity score from 0 to 1 based on five factors:

- **Word count:** Longer questions score higher
- **Number of entities:** More named entities = more complex
- **Intent type:** RELATIONAL and HYBRID score highest
- **Graph requirement:** If the intent classifier flagged requires_graph = true, big boost
- **Time range:** Questions with temporal elements score higher

Score below 0.35 = FAST path. Score above 0.65 = RESEARCH path. In between = depends on entity count.

6. Retrieval — Finding the Right Information

Once the route is decided, the system goes to find information. This is not a simple Google search. It is a multi-step process designed to maximise both the coverage and quality of what gets retrieved.

Step 1: Query Expansion

Instead of searching with just your original question, the system generates 2-4 alternative phrasings using an LLM. For example:

- **Original:** What is the relationship between Pfizer and BioNTech?
- **Expansion 1:** Pfizer BioNTech partnership mRNA vaccine
- **Expansion 2:** BioNTech collaboration pharmaceutical
- **Expansion 3:** Pfizer BioNTech agreement COVID-19

All phrasings are searched simultaneously. Results are deduplicated. This gives much broader coverage than a single search.

Step 2: Document Processing

Each retrieved webpage is fetched and processed:

- **Chunking:** Long articles are split into smaller pieces (chunks) of around 512 tokens each. This is important because LLMs cannot process an entire 5000-word article at once.
- **Embedding:** Each chunk is converted into a vector — a list of 384 numbers — using a sentence-transformers model. These numbers capture the meaning of the text mathematically.
- **Indexing:** All vectors are stored in FAISS, a fast similarity search library. This allows the system to instantly find the most semantically similar chunks to any query.

Step 3: Ranking

Documents are ranked by combining three signals:

- **Relevance score:** How semantically similar is the document to the query?
- **Credibility score:** Is this a reputable source?
- **Recency score:** How recently was this published?

The top-ranked documents proceed to the next stage.

---

7. Knowledge Graph Construction — Building the Relationship Map

This is where the system does something most search tools cannot. Instead of just finding relevant documents, it reads them to understand who is connected to who, what happened when, and how things relate to each other.

Think of it like this: after reading 10 articles about vaccines, a human researcher would naturally start to see patterns — Pfizer partnered with BioNTech, BioNTech licensed technology from Moderna's patent pool, Pfizer contracted Lonza for manufacturing. The knowledge graph captures all of these connections in a structured way.

Step 1: Entity Extraction

The system reads the top 8 documents and uses a combination of two methods to find all named entities:

- **spaCy NER:** A natural language processing model that identifies people, organisations, locations, products, and dates in text. Fast and reliable for common entity types.
- **LLM Extraction:** For more nuanced entities that spaCy might miss, an LLM call extracts additional entities with aliases and attributes.

Duplicate entities are merged. If 'Pfizer Inc', 'Pfizer', and 'PFE' all appear, they become one entity with aliases.

Step 2: Relationship Extraction

For each document, the system extracts relationship triples in the format: Subject → Predicate → Object. For example:

- Pfizer → partnered with → BioNTech
- BioNTech → developed → mRNA vaccine

- Lonza → manufactures for → Moderna

This is done using both rule-based pattern matching (for common relationship patterns like 'X founded Y' or 'X is CEO of Y') and LLM extraction (for complex relationships the rules miss).

Step 3: Graph Construction

All entities become nodes in the graph. All relationships become edges connecting those nodes. The graph is stored in NetworkX, a Python graph library, as a directed multigraph — meaning edges have direction (A relates to B is different from B relates to A) and multiple edges can exist between the same two nodes.

Step 4: Subgraph Extraction

The full graph might have 50-100 nodes after processing 8 documents. For any given query, only a relevant portion is needed. The system identifies which entities from the query exist in the graph, then extracts all entities within 2 hops of those central entities. This subgraph is what gets used for synthesis.

---

8. Temporal Decay — Old Information Should Count for Less

This is one of the new enhancements added to the original system. The problem it solves is simple: a relationship extracted from a 2019 article should not be trusted the same as one from last week — especially for questions about the current state of things.

How It Works

Every relationship edge in the knowledge graph gets a freshness score. The formula is:

> **Formula:** freshness = e^(−age_in_days ÷ 90)

The number 90 is the half-life — meaning a relationship that is 90 days old scores exactly 0.5. Here is what different ages score:

Age of Relationship	Freshness Score
7 days old (last week)	0.93 — almost full trust
30 days old (last month)	0.79 — still fresh
90 days old (3 months)	0.50 — half trust
180 days old (6 months)	0.25 — low trust
365 days old (1 year)	0.06 — very low trust
Unknown date	0.70 — neutral, not penalised

Where Does the Date Come From?

The system looks for dates in this order of priority:

- **1st choice:** The date mentioned in the relationship itself — for example if the text said 'In 2023, Pfizer announced...' then 2023 is used
- **2nd choice:** The publication date of the source article
- **3rd choice:** The date the system retrieved the article
- **No date found:** A neutral score of 0.70 is assigned

How It Affects the Answer

The freshness score is blended into the relationship's confidence score at 30% weight. So a relationship that originally had 0.9 confidence but is 6 months old gets adjusted down. Relationships older than 1 year get a staleness warning flag. These stale relationships are included in the graph but ranked lower during synthesis.

---

9. Graph Scorer — Smarter Entity Ranking

The original system ranked entities in the knowledge graph by a very basic method: how many words in the entity name appeared in the query. So if you asked about 'Pfizer vaccine distribution', any entity with those words scored higher.

This is essentially string matching. It completely ignores the structure of the graph — which entities are actually important based on their connections, not just their name.

The new graph scorer replaces this with two proper graph algorithms.

Algorithm 1: Weighted PageRank

PageRank is the same algorithm Google originally used to rank web pages. The idea is: an entity that many other entities point to must be important. If 15 different relationships all connect to Pfizer, Pfizer is clearly a central node.

In this system, PageRank is weighted — meaning edges with higher confidence and higher freshness count for more. A high-confidence, recent relationship pointing to an entity gives that entity more PageRank score than a low-confidence, old relationship.

Algorithm 2: Betweenness Centrality

Betweenness centrality measures something different. It finds entities that act as bridges between different parts of the graph. For multi-hop reasoning questions — like 'how did Pfizer's BioNTech partnership affect Lonza's manufacturing capacity' — the bridge entity (BioNTech, in this case) is often the most important one to include in the answer even if it appears less frequently.

An entity that lies on the shortest path between many pairs of other entities gets a high betweenness score.

The Composite Score

Both algorithms are combined into one composite score per entity:

- **40%:** PageRank — structural importance
- **25%:** Betweenness centrality — bridge importance
- **20%:** Query match — does the entity name appear in the query?
- **10%:** Extraction confidence — how sure was the system when it found this entity?

- **5%:** Mention count — how many documents mentioned this entity?

Entities are re-ranked by this composite score. The top 50 entities go forward to synthesis. This means structurally important entities surface even if their names do not literally appear in the query.

---

10. Cross-Session Memory — The System Remembers

This is another major new addition. The original system started completely fresh every single time you asked a question. There was a 'session_id' field in the code but it was never actually used.

The Memory Manager changes this. It stores what the system learned from each session and injects that knowledge into future related queries.

What Gets Saved After Each Query

- **Session record:** The query text, a 500-character summary of the answer, the quality score, and which execution path was used
- **Entity facts:** Every named entity found in the session — companies, people, places, products
- **Relationship facts:** Every relationship triple extracted from the knowledge graph, with confidence scores

How Retrieval Works

When a new query comes in, the memory manager searches past sessions using two methods:

- **Entity overlap:** If your new query mentions Pfizer and past sessions also had Pfizer, those sessions are retrieved
- **Keyword overlap:** The top 8 meaningful words from your query are searched against past query text

The most relevant past sessions (ranked by quality score) are retrieved. Their context — past queries, answer summaries, known relationship facts — is assembled into a memory block and prepended to the current evidence before synthesis.

What the Answer Generator Sees

Before writing your answer, the LLM sees something like this at the top of its context:

> **[MEMORY CONTEXT FROM PAST SESSIONS]** Related past queries: 'How did Pfizer partner with BioNTech?' | Known facts: Pfizer → partnered with → BioNTech (confidence: 0.92), BioNTech → developed → mRNA vaccine technology (confidence: 0.88) | Past answer summary: Pfizer and BioNTech formed a partnership in 2020 to develop COVID-19 vaccines using BioNTech's mRNA platform...

This means the system builds on its own research over time. The more you use it on a topic, the better its answers get.

Cleanup

Memory entries expire after 30 days by default (configurable). Old sessions are automatically deleted to keep the database lean.

---

11. Speculative RAG — Three Answers, One Winner

This is the most significant architectural change in the entire system. Understanding it requires understanding the core weakness of standard RAG first.

The Problem With Standard RAG

In a normal RAG system, this happens: retrieve documents → build evidence → generate one answer. If the retrieval was slightly off — maybe the wrong documents were picked up, or the most important relationship was missed — you get a bad answer. There is no fallback. You only find out the answer was bad after reading it.

The Speculative RAG Solution

Instead of generating one answer, the system now generates three answers simultaneously, each using a different way of looking at the same retrieved documents. Think of it like asking three different experts to answer the same question independently, then comparing their answers.

Strategy	What It Does
BROAD — Wide Coverage	Uses the top 10 documents ranked by overall relevance score. Maximises how much ground is covered. Good for comprehensive factual questions.
ENTITY — Entity Focused	Re-ranks documents by how many query entities they mention. If you asked about Pfizer and BioNTech, documents that actually discuss Pfizer and BioNTech get boosted. Good for entity-specific questions.
RELATIONAL — Graph Focused	Uses documents that are sources of high-confidence relationships in the knowledge graph. Also builds 2-hop relationship chains. Good for connection and dependency questions.

All Three Run at the Same Time

The three strategies use a thread pool to run simultaneously. On a cloud LLM like Groq, all three answers are generated in roughly the same time it takes to generate one. On a local model like Ollama, they run one after another (since the local model can only handle one request at a time anyway).

The LLM Judge Picks the Winner

Once all three answers are ready, a judge LLM scores each one on four dimensions:

- **Grounding (35% weight):** Are all the claims in this answer actually supported by the evidence? Or is the system making things up?
- **Completeness (25% weight):** Does the answer fully address what was asked, or does it miss major aspects?
- **Factuality (25% weight):** Are the specific facts accurate and verifiable?
- **Relevance (15% weight):** Does the answer stay on topic throughout?

The answer with the highest weighted composite score wins. Ties are broken by answer confidence score, and then by preferring the RELATIONAL strategy over ENTITY over BROAD.

What Happens to the Losers

The two losing candidates are not discarded. They are stored in the pipeline state metadata. This data can be used to:

- **Debugging:** If the answer seems wrong, you can inspect which strategy won and why the others lost
- **Fine-tuning:** The candidate scores are training signal — you now have labelled examples of better and worse answers for the same query
- **Audit trail:** Every answer has a paper trail showing what alternatives existed

---

12. Uncertainty Quantification — Honest About What It Doesn't Know

The original system had a confidence number (like 0.73) attached to each claim, but this number was hidden in metadata. The answer text itself gave no indication of which specific facts were well-supported and which were shaky.

The uncertainty quantifier changes this by computing a confidence band for every claim and injecting warning markers directly into the answer text.

How the Confidence Band Is Computed

Each claim gets a range like (0.58, 0.88) instead of just 0.73. The width of this range depends on four factors:

- **Source count:** A claim supported by zero sources gets a very wide band (high uncertainty). A claim supported by 3 or more sources gets a narrow band (lower uncertainty).
- **Source diversity:** If all sources supporting a claim come from the same website or domain, that is less reliable than sources from three different domains. Low diversity widens the band.
- **Controversy flag:** If the claim is marked as controversial during extraction, the band gets wider automatically.
- **LLM factuality score:** The LLM judge's factuality score is blended in at 20% weight to adjust the confidence up or down.

What Gets Flagged in the Answer Text

Claims are classified into three levels:

- **Low uncertainty (confidence above 0.70):** No marker added. The claim appears normally in the answer.
- **Medium uncertainty (confidence 0.45 to 0.70):** The marker [uncertain — limited sources] is injected before the citation number.
- **High uncertainty (confidence below 0.45):** The marker [uncertain — very limited sources] is injected.

> **Example:** Before: 'OpenAI was valued at $29 billion [1]' After: 'OpenAI was valued at $29 billion [uncertain — limited sources] [1]'

---

13. Quality Evaluation — The LLM Judge

After the answer is generated, the system evaluates it before returning it to you. This evaluation has two layers.

Layer 1: Heuristic Metrics (Fast, No LLM Needed)

- **Citation coverage:** What fraction of sentences in the answer have a citation? Answers with many uncited claims score low.
- **Grounding score:** How much word overlap is there between the answer and the extracted claims? Low overlap suggests hallucination.
- **Coherence score:** Does the answer have a reasonable structure? Is it repetitive?
- **Completeness score:** Is the answer long enough and does it reference enough sources?
- **Source diversity:** Does the answer cite sources from multiple different domains?

Layer 2: LLM Judge (Deep, Requires LLM)

Four separate LLM calls evaluate the answer on grounding, factuality, relevance, and completeness. These are more nuanced than the heuristic metrics — an LLM can actually read the answer and check if specific claims are supported by the evidence.

In the original system, these four calls ran one after another. In the updated system, they run in parallel, cutting evaluation time by roughly 70% on cloud LLMs. On local Ollama, they run sequentially to avoid timeout errors.

Judge scores are cached by answer text hash. If self-healing produces the same answer, the evaluation is retrieved from cache instead of re-running four LLM calls.

The Composite Score

The final quality score blends heuristic metrics (60% weight) and LLM judge scores (40% weight) into a single number. This composite score is compared against a threshold. If it passes, the answer goes out. If it fails, self-healing kicks in.

---

14. Self-Healing — Automatic Retry When Quality Fails

If the quality evaluation finds problems, the system does not just return a bad answer. It diagnoses which specific metric failed and applies the right fix automatically.

Problem Detected	Healing Strategy Applied
Not enough sources (low completeness)	EXPAND-QUERY: Add entity names and domain terms to the search, retrieve more documents
Claims not supported by evidence (low grounding)	RE-RETRIEVE: Search again with different parameters, get fresh sources
Too few citations in answer text (low citation coverage)	REFINE-SYNTHESIS: Regenerate the answer from the same evidence with instructions to add more citations
Simple question got a poor answer	SWITCH-PATH: Escalate from fast path to full research path

The system tries up to 2 healing attempts before giving up and returning the best answer it has. A thread-safety fix was also applied here — the original code mutated a shared settings variable (max_search_results) which would break under concurrent requests. The fix passes the override as a local parameter instead.

15. The Cache — Serving Repeat Queries Instantly

The cache is how the system avoids re-running the full 12-30 second pipeline every time someone asks a similar question. The original cache only matched exact query text — rephrase the question even slightly and the full pipeline ran again.

The Original Cache (Exact Match)

Query text → MD5 hash → lookup in cache. If the exact hash exists, return the cached answer. If not, run the full pipeline. Simple but limited.

The New Semantic Cache (Meaning Match)

When a query is processed and cached, the system also embeds the query text into a vector using sentence-transformers. This vector is stored alongside the cache key.

When a new query comes in:

- **Step 1:** Check for exact hash match first (instant, free)
- **Step 2:** If no exact match, embed the new query and compare it against all stored query vectors using cosine similarity
- **Step 3:** If similarity is above 0.92 (very close in meaning), return the cached answer

> **Result:** 'What is the relationship between OpenAI and Microsoft?' and 'How are OpenAI and Microsoft connected?' both return the same cached answer — even though the words are completely different.

Tiered Cache Expiry

Not all answers should be cached for the same amount of time. The new cache sets expiry based on question type:

Question Type	Cache Duration
SEMANTIC — definitions, explanations	24 hours — these facts do not change quickly
ENTITY — facts about a specific thing	6 hours — entity facts are fairly stable
RELATIONAL — connections between things	2 hours — relationships can change
TEMPORAL — time-sensitive queries	15 minutes — these need fresh data
HYBRID — mixed queries	1 hour — balanced

Component-Level Caching

Beyond full pipeline responses, the cache now stores intermediate results:

- **Entity extraction results:** Per document content hash — if the same article appears in two different queries, its entities are not re-extracted
- **Relationship extraction results:** Same — expensive LLM calls per document are not repeated
- **LLM Judge scores:** Per answer text hash — same answer text is not re-evaluated
- **Speculation candidates:** Per strategy + query — if the same query hits again, cached candidates are reused

16. Regression Detection — Is the System Getting Worse?

Every query's quality metrics are saved to a SQLite database. Over time, the system builds up a historical record of its own performance. Regression detection uses this history to notice when quality is dropping.

How It Works

After every 10 queries or so, the system calculates baseline statistics (mean and standard deviation) for each quality metric from the last 100 evaluations. For each new answer, it computes a Z-score:

> **Formula:** Z-score = (baseline_mean − current_value) ÷ baseline_standard_deviation

A Z-score above 2.0 means the current answer is more than 2 standard deviations below the historical average — that is a regression.

Severity Levels

Z-Score	Severity
2.0 to 3.0	LOW — monitor but no action needed
3.0 to 4.0	HIGH — investigate recent changes
Above 4.0	CRITICAL — something is seriously wrong

Multiple HIGH or CRITICAL alerts in a short period can trigger self-healing on subsequent queries. Each alert also includes a specific recommendation — for example, 'LLM factuality concerns — verify source credibility' or 'Citation coverage dropped — check if retrieval quality degraded'.

17. Output Guardrails — The Final Check

Before the answer reaches you, it goes through one final set of checks. These are different from the input guardrails — instead of checking the question, they check the answer.

- **Citation coverage:** The answer must have citation markers ([1], [2], etc.) covering a minimum fraction of its claims. If not, it is sent back for self-healing.
- **Confidence threshold:** The answer's overall confidence must exceed the minimum threshold (default 0.5).
- **Grounding:** The grounding score must be above 0.6. Answers that appear to be making things up are rejected.
- **PII check:** The answer text is scanned for patterns like social security numbers or credit card numbers. If found, the answer is blocked.
- **System prompt leakage:** The answer is checked for signs that internal system instructions leaked into the output.

18. Complete Architecture Summary

Component	Technology Used
Web Framework / API	FastAPI with Pydantic validation
LLM (Cloud)	Groq — Llama 3.3 70B Versatile
LLM (Local)	Ollama — Llama 3.1 8B
Embeddings	sentence-transformers all-MiniLM-L6-v2
Vector Search	FAISS (Facebook AI Similarity Search)
Named Entity Recognition	spaCy en_core_web_sm / en_core_web_trf
Knowledge Graph	NetworkX MultiDiGraph
Graph Algorithms	NetworkX PageRank + Betweenness Centrality
Cache (Disk)	diskcache Python library
Cache (Semantic Index)	sentence-transformers + cosine similarity
Memory Store	SQLite with WAL mode
Evaluation Store	SQLite
Parallelism	Python ThreadPoolExecutor
Configuration	Pydantic Settings + .env file
Logging	Loguru structured logging

19. Before and After — What Changed

Original System	Enhanced System
Single answer generated per query	3 parallel candidates, LLM judge picks best
Entity ranking by word overlap (string matching)	Weighted PageRank + betweenness centrality
All graph edges treated equally regardless of age	Exponential temporal decay — old edges downweighted
Every session starts completely fresh	Cross-session SQLite memory with context injection
Claim confidence hidden in metadata	Uncertainty bands + inline markers in answer text
LLM judge runs 4 calls sequentially	4 calls in parallel — 70% faster on cloud LLMs
Cache matches exact query text only	Semantic similarity cache — paraphrases hit cache

Original System	Enhanced System
Uniform 1-hour cache TTL for everything	Tiered TTL: 15 min to 24 hours by intent type
use_cache API param ignored	Properly wired through the full pipeline
Settings mutated during self-healing (thread-unsafe)	Local parameter override — thread safe

End of Document

Pranjal Raghava — ValueMatrix AI